

Backend Design and API Specification

1. Purpose

This document sets out the backend approach for Roomify.Ai app. It covers the technology choices, the 3D model format decision, the database design, and the API that the frontend and the Unity environment will call..

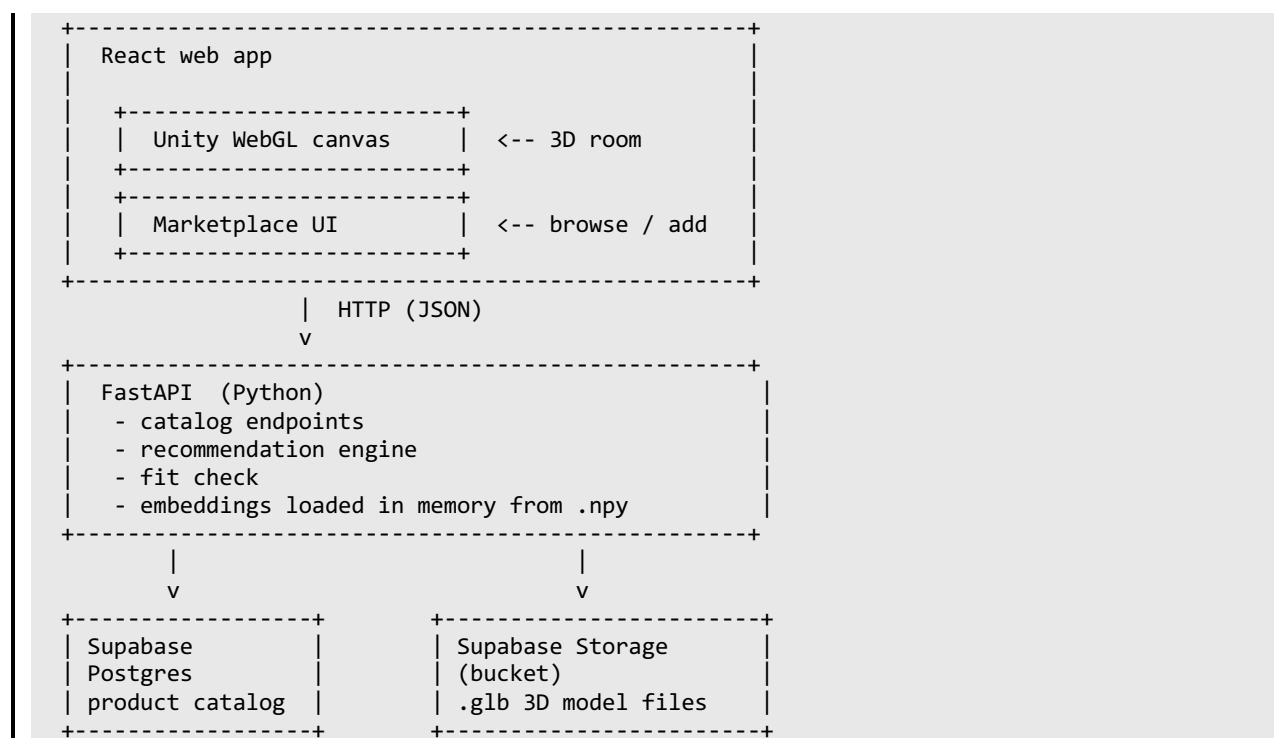
2. What the backend has to do

The backend is responsible for four things for now as per my planning:

- Hold the product catalog and serve it to the marketplace (browse, filter, search).
- Run the recommendation engine (need-first ranking, taste as tiebreaker).
- Run the fit check (does this item physically fit in the scanned room).
- Tell Unity what 3D model need to add.

3. System architecture

Three pieces, clearly separated:



Unity downloads the model file directly from the storage bucket. It does not stream through our API. The API only hands over the URL. This keeps our server light and lets the storage CDN do the heavy lifting.

4. Technology choices

Each choice below was made against the alternatives, not by default.

Layer	Chosen	Reason
API framework	FastAPI (Python)	The recommender is Python (pandas, sentence-transformers). A Python API means no rewrite and no bridge between languages.
Database	Postgres, hosted on Supabase	The catalog is tabular with heavy filtering by room, category, size band and score. That is SQL-shaped data.
3D model storage	Supabase Storage bucket	Model files are binary and large. They do not belong inside a database. The database stores only the URL.
Embeddings	Pre-computed NumPy (.npy) file, loaded into memory at startup	360 vectors is a tiny amount of data. Holding them in memory is faster and simpler than a database lookup.
Model format	GLB (converted from FBX)	Unity cannot read FBX at runtime. GLB can be loaded on demand and also works directly in the browser.

4.1 Options considered and rejected

Rejected	Why not
Firebase / Firestore	Its database is NoSQL (documents). Our catalog is rows and columns with heavy filtering. Forcing tabular data into a document store makes queries awkward for no gain.
SQLite	Genuinely simplest, and fine for local work. But it lives on the API server's disk, so the moment the frontend needs a live hosted URL we would have to migrate anyway. We skip that migration by starting on Supabase.
Next.js	Its main advantage is bundling a backend with the frontend. Our backend must be Python regardless, so that advantage disappears.

5. The 3D model format decision

Our source models are FBX. This is the single most important technical decision in the document, so it is set out in full.

5.1 The problem

Unity reads FBX in the editor, at build time. It does not read FBX at runtime. Once Unity is compiled to WebGL, there is no FBX reader inside the build.

Our product flow requires runtime loading. The user picks a sofa from the marketplace after the app is already running. That model was not known when Unity was built. Therefore FBX cannot work in our flow.

5.2 The decision

Convert FBX to GLB once, as a batch job in Blender. Store the GLB files in the Supabase bucket. Keep the FBX files as the master source; they are not deleted.

6. Data model

One main table. It mirrors the catalog we already built. (This is anticipated schema, we can change as per requirements)

products		
id	text	primary key
name	text	
category	text	chair, sofa, table, ...
room_type	text	living room, bedroom, office
width_mm	integer	
depth_mm	integer	
height_mm	integer	
size_band	text	S M L (within category)
footprint_mm2	integer	width * depth
storage_score	integer	0-100, percentile ranked
is_storage	boolean	room-aware indicator
description	text	used for taste matching
image_url	text	
model_url	text	link to the .glb in the bucket
embedding_index	integer	row number in the .npy file

Two points worth noting:

- model_url is a text link, not the file. The file lives in the storage bucket.
- embedding_index maps a product row to its vector in the .npy file. This is how the database and the in-memory embeddings stay in step without storing vectors in Postgres.

7. API specification

7.1 Conventions

- All responses are JSON.
- All dimensions are in millimetres, everywhere, with no exceptions.
- Standard HTTP codes: 200 success, 404 not found, 422 bad input.
- Read operations are GET. Operations that send a room description are POST, because the room does not fit sensibly in a URL.

7.2 GET /health

Confirms the service is up and the embeddings loaded. Also used as the keep-alive ping that stops the free Supabase project from pausing.

```
{ "status": "ok", "products": 360, "embeddings_loaded": true }
```

7.3 GET /categories

Powers the marketplace filter menu. Returns the 10 categories and how many products sit in each.

```
[  
  { "category": "chair", "count": 48 },  
  { "category": "sofa", "count": 36 },  
  ...  
]
```

7.4 GET /products (the marketplace list)

The main browse endpoint. Every filter is optional; with no filters it returns the first page of everything.

Parameter	Type	Meaning
category	text	Filter to one category, e.g. sofa.
room_type	text	Filter to items suited to a room, e.g. bedroom.
size_band	S M L	Filter by size band within the category.
max_width_mm	integer	Exclude anything wider than this. Used when the room is tight.
search	text	Free text match on the product name.
limit	integer	Page size. Default 20.
offset	integer	Page start. Default 0.

Response:

```
{  
  "total": 36,  
  "limit": 20,  
  "offset": 0,  
  "items": [  
    {  
      "id": "B07XYZ123",  
      "name": "Two Seater Fabric Sofa",  
      "category": "sofa",  

```

```

    "size_band": "M",
    "width_mm": 1620,
    "depth_mm": 880,
    "height_mm": 850,
    "image_url": "https://.../thumb.jpg"
  }
]
}

```

The list response deliberately leaves out `model_url` and `description`. A marketplace grid does not need them, and sending 20 model links per page wastes bandwidth we are metered on.

7.5 GET /products/{id} (a single product)

Called when the user clicks a product, and again when Unity needs to place it. This is the endpoint that carries the 3D model link.

```

{
  "id": "B07XYZ123",
  "name": "Two Seater Fabric Sofa",
  "category": "sofa",
  "room_type": "living room",
  "size_band": "M",
  "width_mm": 1620,
  "depth_mm": 880,
  "height_mm": 850,
  "footprint_mm2": 1425600,
  "storage_score": 22,
  "is_storage": false,
  "description": "Mid century two seater in grey fabric ...",
  "image_url": "https://.../full.jpg",
  "model_url": "https://.../models/B07XYZ123.glb"
}

```

Returns 404 if the id does not exist.

7.6 POST /recommend

The two-stage engine we already built. Stage one filters on geometry (does it fit in the room, given what is already there). Stage two ranks what survives, weighting need at 0.7 and taste at 0.3.

Request:

```

{
  "room": {
    "width_mm": 3400,
    "length_mm": 4200,
    "height_mm": 2400,
    "room_type": "bedroom"
  },
  "existing_furniture": [
    { "width_mm": 1400, "depth_mm": 2000 }
  ],
  "taste": "warm wood, minimal, scandinavian",
  "limit": 10
}

```

Response:

```

{
  "free_floor_mm2": 11480000,
  "items": [
    {
      "id": "B08ABC456",
      "name": "Oak Bedside Table",
      "score": 0.83,

```

```

    "need_score": 0.91,
    "taste_score": 0.64,
    "reason": "high storage for a bedroom, fits remaining floor space",
    "width_mm": 450,
    "depth_mm": 400,
    "height_mm": 550,
    "model_url": "https://.../models/B08ABC456.glb"
  }
]
}

```

The reason field matters. A recommendation the user cannot understand is a recommendation they will not trust, and it is the thing a funding panel will ask about.

7.7 POST /fit-check

The student-moving case from notebook 03. Given a room and a list of items the user wants to bring, say what fits and what has to go.

```

{
  "room": { "width_mm": 2800, "length_mm": 3200 },
  "items": ["B07XYZ123", "B08ABC456", "B09DEF789"]
}

{
  "fits": false,
  "floor_needed_mm2": 9950000,
  "floor_available_mm2": 8960000,
  "drop_suggestion": ["B07XYZ123"],
  "fits_after_drop": true
}

```

8. End-to-end flow: adding furniture from the marketplace

This is the flow the whole design exists to serve.

1. App opens. Unity WebGL loads the scanned room. Marketplace sits below the 3D view.
2. React calls `GET /products?room_type=bedroom`
-> marketplace grid fills with thumbnails.
3. User clicks a sofa.
React calls `GET /products/B07XYZ123`
-> gets real dimensions + `model_url`.
4. React hands that JSON to Unity
via `unityInstance.SendMessage()`.
5. Unity downloads the `.glb` from `model_url`
(straight from the bucket, not through our API),
loads it with `glTFast`,
rescales it to the real mm dimensions,
and places it in the room.
6. User drags it around. Unity handles this alone.
7. User deletes an item. Unity handles this alone.
No backend call needed.

Steps 6 and 7 need no backend at all. That is the point of the split: once the model and its true size are handed over, the room is Unity's problem and the catalog is ours.

9. Constraints and risks

Risk	Detail	Mitigation
Storage cap	The free Supabase tier gives 1 GB of file storage. If our GLB files average 3 MB, 360 of them will not fit.	Measure the exported GLB sizes before committing. Compress meshes and textures. If it still does not fit, move the bucket to Cloudflare R2, whose free tier is 10 GB. The database stays on Supabase either way, since only the <code>model_url</code> changes.
Project pausing	Free Supabase projects pause after 7 days with no requests. During a funding demo, a paused project means a dead app.	A scheduled ping to <code>GET /health</code> every few days keeps it awake. Set this up now, not the week of the demo.
Bandwidth	5 GB of egress per month on the free tier. 3D models are the heaviest thing we serve.	Serve models through the storage CDN, which does not count against the quota, rather than direct storage URLs.
Dataset licence	The catalog is built on Amazon Berkeley Objects, licensed CC BY-NC 4.0. Non-commercial only.	Fine for a prototype and a funding demo. It cannot ship commercially. A real supplier catalog has to replace it before launch, and this should be stated openly rather than discovered later.
Missing dimensions	307 of 360 products have full dimensions. The rest do not.	Products without dimensions are excluded from fit checks and geometric filtering. They can still be browsed.

10. Next steps

In order:

- Measure the FBX files: how many, and how large once converted to GLB. This decides Supabase Storage versus Cloudflare R2.
- Write the Blender batch script to convert FBX to GLB.
- Create the Supabase project and the products table.
- Write a loader that pushes catalog.csv into the table and the model files into the bucket.
- Build the FastAPI service, endpoints in the order listed in section 7.
- Set up the keep-alive ping.
- Agree the SendMessage contract with the Unity developer, so both sides expect the same JSON.

The last point is the one most likely to be forgotten and the most likely to cost a day. It should be agreed in writing before either side builds against it.